

Practical 1

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>

using namespace std;
using namespace std::chrono;

// Bubble Sort
void bubbleSort(vector<int> &arr)
{
    int n = arr.size();

    for (int i = 0; i < n - 1; i++){
        for (int j = 0; j < n - i - 1; j++){
            if(arr[j] > arr[j + 1]){
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}

// Selection Sort
void selectionSort(vector<int> &arr)
{
    int n = arr.size();

    for (int i = 0; i < n - 1; i++){
        int min = i;
        for (int j = i + 1; j < n; j++){
            if(arr[j] < arr[min]){
                min = j;
            }
        }
        swap(arr[i], arr[min]);
    }
}

// Insertion Sort
void insertionSort(vector<int> &arr)
{

```

```

int n = arr.size();

for (int i = 1; i < n; i++){
    int key = arr[i];
    int j = i - 1;

    while (j >= 0 && arr[j] > key){
        arr[j + 1] = arr[j];
        j--;
    }
    arr[j + 1] = key;
}

// Merge Function
void merge(vector<int> &arr, int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;

    vector<int> L(n1), R(n2);

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2){
        if (L[i] <= R[j]){
            arr[k++] = L[i++];
        }else{
            arr[k++] = R[j++];
        }
    }

    while (i < n1)
        arr[k++] = L[i++];

```

```

        while (j < n2)
            arr[k++] = R[j++];
    }

// Merge Sort
void mergeSort(vector<int> &arr, int left, int right)
{
    if (left < right){
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

// Partition using First Element as Pivot
int partition(vector<int> &arr, int low, int high)
{
    int pivot = arr[low];
    int i = low + 1;
    int j = high;

    while (true){
        while (i <= high && arr[i] <= pivot)
            i++;

        while (arr[j] > pivot)
            j--;

        if (i < j){
            swap(arr[i], arr[j]);
        }else{
            break;
        }
    }

    swap(arr[low], arr[j]);

    return j;
}

```

```

// Quick Sort
void quickSort(vector<int> &arr, int low, int high)
{
    if (low < high){
        int p = partition(arr, low, high);

        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}

int main()
{
    int n = 10000;

    vector<int> arr(n), temp;

    srand(time(0));

    for (int i = 0; i < n; i++)
        arr[i] = rand() % 10000;

    cout << "Number of Elements : " << n << endl << endl;

    auto start = std::chrono::high_resolution_clock::now();
    temp = arr;
    bubbleSort(temp);
    auto stop = std::chrono::high_resolution_clock::now();
    cout << "Bubble Sort: "
        << std::chrono::duration_cast<std::chrono::milliseconds>(stop -
start).count()
        << " ms" << endl;

    start = std::chrono::high_resolution_clock::now();
    temp = arr;
    selectionSort(temp);
    stop = std::chrono::high_resolution_clock::now();
    cout << "Selection Sort: "
        << std::chrono::duration_cast<std::chrono::milliseconds>(stop -
start).count()
        << " ms" << endl;

    start = std::chrono::high_resolution_clock::now();

```

```

temp = arr;
insertionSort(temp);
stop = std::chrono::high_resolution_clock::now();
cout << "Insertion Sort: "
    << std::chrono::duration_cast<std::chrono::milliseconds>(stop -
start).count()
    << " ms" << endl;

start = std::chrono::high_resolution_clock::now();
temp = arr;
mergeSort(temp, 0, n - 1);
stop = std::chrono::high_resolution_clock::now();
cout << "Merge Sort: "
    << std::chrono::duration_cast<std::chrono::milliseconds>(stop -
start).count()
    << " ms" << endl;

start = std::chrono::high_resolution_clock::now();
temp = arr;
quickSort(temp, 0, n - 1);
stop = std::chrono::high_resolution_clock::now();
cout << "Quick Sort: "
    << std::chrono::duration_cast<std::chrono::milliseconds>(stop -
start).count()
    << " ms" << endl;

return 0;
}

```